

Standard Operating Procedure (SOP): Penilaian Kerentanan (Vulnerability Assessment)

Sistem Informasi Kegiatan dan Anggaran Politeknik (SIGAP) - Universitas/Politeknik Negeri

No. Dokumen	SOP-IT-SIGAP-022	NAMA INSTITUSI
Tgl Berlaku	11 Juni 2026	Politeknik Negeri Jakarta (PNJ)
Status Revisi	01 (Pertama)	Departemen / Unit
Halaman	1 dari 5	Unit Pelayanan Sistem Informasi / IT

1. TUJUAN

Tujuan dari prosedur ini adalah:

- Mengidentifikasi kelemahan atau kerentanan dalam sistem, jaringan, dan aplikasi SIGAP-Laravel yang dapat dieksploitasi oleh pihak yang tidak berwenang.
- Mengembangkan, merencanakan, dan menerapkan tindakan perbaikan (mitigasi) untuk mengatasi kerentanan yang ditemukan guna mencegah eksploitasi di masa depan dan menjamin integritas data anggaran/kegiatan kampus.

2. CAKUPAN

Prosedur ini mencakup seluruh aset data, infrastruktur server (Supabase/PostgreSQL), API endpoint, source code Laravel 11, UI React/Inertia.js, serta konfigurasi environment variables sistem SIGAP-Laravel.

3. DEFINISI

Penilaian Kerentanan (*Vulnerability Assessment*) adalah proses sistematis untuk mengidentifikasi, menganalisis, dan mengevaluasi kelemahan/celah keamanan dalam sistem dan aplikasi yang dapat mengancam kerahasiaan (*confidentiality*), integritas (*integrity*), dan ketersediaan (*availability*) data informasi. Proses ini melibatkan penggunaan alat otomatis (*automated tools*), analisis kode statis, serta penilaian risiko untuk mengurangi potensi ancaman.

4. DOKUMEN TERKAIT

Dalam pelaksanaan prosedur ini, dokumen-dokumen berikut wajib dilampirkan/disiapkan:

- Rencana Penilaian Kerentanan (Jadwal & Lingkup)
- Laporan Hasil Penilaian Kerentanan (Hasil scan & tingkat risiko)
- Dokumentasi Temuan Celah Keamanan
- Rencana Tindakan Perbaikan (Mitigasi)
- Kebijakan dan Prosedur Keamanan Informasi
- Catatan Pemantauan dan Tindak Lanjut

5. RINCIAN PROSEDUR

No.	Kegiatan / Langkah Kerja	Tanggung Jawab
5.1	Menyusun Rencana Penilaian Kerentanan Menentukan jadwal pelaksanaan, metodologi yang digunakan, serta alat bantu (<i>tools</i>) yang akan dipakai.	Tim Keamanan IT / Administrator
5.2	Mengidentifikasi Kerentanan Menggunakan alat pemindai otomatis dan pengujian manual untuk mengidentifikasi celah keamanan pada sistem, jaringan, database, dan aplikasi.	Tim Keamanan IT / Administrator
5.3	Menilai & Menganalisis Kerentanan Menganalisis kerentanan yang ditemukan untuk menentukan tingkat risiko (Low, Medium, High, Critical) serta memprioritaskan perbaikan.	Tim Keamanan IT / Administrator
5.4	Pencatatan & Dokumentasi Temuan Mendokumentasikan seluruh temuan celah keamanan secara detail beserta rekomendasi mitigasi/solusi teknis.	Tim Keamanan IT / Administrator
5.5	Kolaborasi & Tindakan Perbaikan Bekerja sama dengan tim teknis/developer untuk menerapkan perbaikan (patching) dan memantau efektivitas perbaikan tersebut.	Tim Keamanan IT & Tim Developer

ALUR KERJA (FLOWCHART PROSEDUR)

```
graph TD
    A([START]) --> B[Perencanaan Penilaian Kerentanan]
    B --> C[Pengumpulan Informasi]
```

```

C --> D[Identifikasi Kerentanan]
D --> E[Analisis Kerentanan]
E --> F[Penilaian Risiko]
F --> G[Prioritas dan Penanganan]
G --> H[Penerapan Tindakan Perbaikan]
H --> I[Verifikasi dan Uji]
I --> J[Dokumentasi dan Pelaporan]
J --> K[Pemantauan dan Tindak Lanjut]
K --> L([END])

```

Penjelasan Detail Flowchart:

1. **START:** Inisiasi proses penilaian kerentanan rutin (berkala) atau insidental.
2. **Perencanaan Penilaian Kerentanan:** Menentukan ruang lingkup pengujian (misal: modul LPJ, auth endpoint) dan sumber daya yang dibutuhkan.
3. **Pengumpulan Informasi:** Mengumpulkan data teknis sistem seperti versi PHP, dependency Laravel, port server yang terbuka, dan konfigurasi API.
4. **Identifikasi Kerentanan:** Menjalankan perkakas audit (`composer audit` , `npm audit` , atau scanner eksternal) untuk menemukan celah keamanan.
5. **Analisis Kerentanan:** Menganalisis sifat celah keamanan yang ditemukan (apakah berdampak langsung pada database atau hanya lokal).
6. **Penilaian Risiko:** Mengklasifikasikan risiko berdasarkan matriks dampak (misal: CVE-2026-48019 berdampak rendah karena validasi form berlapis).
7. **Prioritas dan Penanganan:** Menyusun skala prioritas tindakan perbaikan (mendahulukan celah Critical & High).
8. **Penerapan Tindakan Perbaikan:** Melakukan update versi pustaka, pengkodean ulang kode rentan, atau pengaktifan middleware keamanan (seperti memulihkan CSRF).
9. **Verifikasi dan Uji:** Melakukan pengetesan ulang (re-testing) via automation tests (`composer test`) untuk memastikan celah tertutup dan tidak merusak fungsi aplikasi lainnya.
10. **Dokumentasi dan Pelaporan:** Menyusun dokumen laporan akhir penilaian kerentanan untuk diserahkan kepada pimpinan IT/Direktur.
11. **Pemantauan dan Tindak Lanjut:** Memantau berkala log aktivitas (`[AUTH_AUDIT]`) untuk mengidentifikasi indikasi serangan susulan.
12. **END:** Selesai.

FORMULIR PENILAIAN KERENTANAN

A. Informasi Umum

1. **Nama Prosedur:** Penilaian Kerentanan Aplikasi SIGAP-Laravel
2. **Deskripsi Singkat:** Prosedur untuk menilai kerentanan source code, pustaka dependensi, API, dan database guna mengidentifikasi risiko keamanan serta menerapkan langkah mitigasi.
3. **Tanggal Pembuatan:** 11 Juni 2026
4. **Tanggal Revisi Terakhir:** 11 Juni 2026
5. **Penyusun Prosedur:**
 - **Nama:** Tim Administrator Keamanan Informasi
 - **Jabatan:** Keamanan Sistem Informasi IT PNJ

B. Tujuan & Lingkup

1. **Tujuan:** Mengidentifikasi dan mengevaluasi kerentanan pada aplikasi SIGAP-Laravel untuk mengurangi risiko kebocoran data anggaran dan meningkatkan ketahanan sistem terhadap serangan eksternal.
2. **Lingkup Pengujian:**
 - ☐ Sistem Jaringan
 - ☒ Aplikasi Web (SIGAP-Laravel Frontend & Backend)
 - ☒ Sistem Operasi / Lingkungan Server (Supabase/PostgreSQL Cloud)
 - ☐ Perangkat Keras
 - ☒ Lainnya: Endpoint API & Dependency Package (Composer & NPM)

C. Prosedur Pelaksanaan

1. Persiapan Penilaian

- **Sistem yang Dinilai:** Database Supabase PostgreSQL, Framework Laravel 11, Frontend React (Inertia.js).
- **Area yang Dicakup:** Modul KAK, Kegiatan, Pencairan Dana, LPJ, dan Autentikasi Pengguna.
- **Alat Penilaian yang Digunakan:**
 - ☒ Alat Pemindai Kerentanan (*Vulnerability Scanner*): `composer audit`, `npm audit`
 - ☒ Alat Pengujian Penetrasi (*Penetration Testing Tools*): PHPUnit Test Cases & Browser Mocking
- **Jadwal Penilaian:**
 - **Tanggal:** Setiap akhir bulan atau sesaat sebelum rilis versi produksi.
 - **Waktu:** 09:00 - 15:00 WIB.

2. Pelaksanaan Penilaian

- **Metode Pengumpulan Data:** Analisis konfigurasi (.env, bootstrap/app.php) dan pemindaian library package.json & composer.json.
- **Dokumentasi Data:** Log output dari audit CLI disimpan ke file `Vulnerability_Analysis_Report.md`.
- **Metode Identifikasi:** Pencocokan kode CVE yang terdaftar di basis data kerentanan global (GitHub Advisory, Mitre CVE).
- **Catatan Temuan Kerentanan:** Teridentifikasi 3 celah pada Symfony components, 1 celah CRLF pada default email validator Laravel (CVE-2026-48019), serta celah keamanan pada pengunggahan file (potensi eksekusi script / malware).
- **Pengujian Penetrasi (Jika Diperlukan):** Pengujian penetrasi input SQLi pada form login dilakukan via PHPUnit di [AuthenticationTest.php](#). Pengujian penetrasi virus EICAR dan unggahan script PHP/JS dilakukan di [LampiranTest.php](#).

3. Analisis dan Evaluasi

- **Klasifikasi Risiko:**
 - Symfony mailer/mime/routing: Risiko **High/Medium** (telah ditambah ke versi terbaru).
 - Laravel framework: Risiko **Low** (telah dimitigasi oleh validasi email custom pada form request).
 - Unggahan berkas berbahaya (virus/script): Risiko **High** (telah dimitigasi dengan filter konten file).
- **Dampak Potensial:** Argument injection, email header injection, SSRF, redirection bypass, dan remote code execution (RCE) via upload malware jika tidak ditambah.

4. Mitigasi dan Tindak Lanjut

- **Langkah-Langkah Mitigasi:**
 1. Menjalankan perintah `composer update` untuk menaikkan patch version Symfony.
 2. Menjalankan `npm audit fix` untuk menambal celah pada library frontend.
 3. Mengaktifkan kembali middleware CSRF global.
 4. Menerapkan pemindaian signature virus EICAR dan script PHP/JS berbahaya pada berkas lampiran anggaran di [LampiranService.php](#).
- **Tanggung Jawab Pelaksana:** IT Security Lead & Backend Developer.
- **Pengawasan:** Koordinator IT Politeknik.
- **Reevaluasi:** Pengujian ulang dilakukan menggunakan perintah `composer test` pasca pembaruan kode.

5. Pelaporan dan Dokumentasi

- **Format Laporan:** Berkas Markdown laporan keamanan.

- **Isi Laporan:** Daftar kerentanan, severity level, status patch, langkah mitigasi, dan rekomendasi berkelanjutan.
 - **Distribusi Laporan:**
 - **Penerima:** Kepala UPT Sistem Informasi PNJ & Wakil Direktur 2 Bidang Keuangan.
 - **Tanggal Pengiriman:** 11 Juni 2026.
-

D. Dokumentasi dan Catatan

1. **Dokumen Penilaian:** Laporan Analisis Kerentanan ([Vulnerability Analysis Report.md](#)).
 2. **Catatan Temuan:** Hasil log audit SQL Injection ([SQL Injection Audit.md](#)).
 3. **Laporan Mitigasi:** Laporan Uji Coba Unit Test Pasca Patch.
 4. **Penyimpanan Dokumentasi:**
 - **Lokasi:** Server Dokumentasi Keamanan Internal (Folder `c:/xampp/htdocs/SIGAP-Laravel/`).
 - **Akses ke Dokumentasi:** Terbatas untuk Tim Administrator dan IT Security saja.
-

E. Persetujuan dan Tanda Tangan

1. **Persetujuan Prosedur:**
 - ☒ Disetujui
 - ☐ Ditolak
 - ☐ Ditunda
2. **Tanda Tangan Persetujuan:**
 - **Nama:** Dr. Ir. Koordinator IT Politeknik, M.T.
 - **Jabatan:** Kepala UPT Sistem Informasi
 - **Tanggal:** 11 Juni 2026